VulnVersion: Graph-Augmented Evidence-Constrained History Reconstruction for Vulnerability-Affected Version Identification

Anonymous Authors

***Abstract*—Identifying which released versions are affected by a disclosed vulnerability is essential for vulnerability management, dependency maintenance, and downstream risk assessment. Existing approaches often infer affected version ranges from advisory text, fixing commits, or vulnerability-inducing commits, but such interval-based reasoning is fragile under backports, release-branch divergence, refactorings, partial fixes, and vulnerability-preserving code movement. This paper formulates CVE affected-version identification as evidence-constrained history reconstruction over Git history. Given a CVE fixing commit and a repository's release history, our key insight is to reconstruct vulnerability-relevant history events and boundary events, propagate vulnerability state through the Git DAG, and project the resulting states to release tags. We present VulnVersion, a graph-backed approach that extracts root-cause anchors and vulnerability/fix predicates from patches, reconstructs branch-specific vulnerability state, and predicts the affected released-version set through Git-DAG-based state propagation. We evaluate VulnVersion on a public benchmark for vulnerability-affected version identification and compare it against advisory-based, commit-range, SZZ-style, patch-similarity, and LLM-only baselines.**

***Index Terms*—software vulnerability, affected-version identification, software evolution, program analysis, large language models**

I. Introduction

Maintainers, vulnerability scanners, and downstream users often need a concrete answer to a simple question: which released versions are affected by this CVE? This answer drives upgrade guidance, dependency triage, emergency patching, and risk assessment. A scanner that misses an affected release leaves users exposed, while a scanner that marks safe releases as affected creates unnecessary upgrade work. Public CVE metadata and advisory ranges are useful starting points, but they are not always aligned with the actual release history of a project. The affected-version identification task therefore asks for a repository-grounded output: for each CVE, predict the affected release tags or affected versions in the official release universe [1]. The desired output is a complete released-version set, not a best-effort advisory interval.

This task is hard because vulnerability fixes rarely map cleanly to a linear version interval. A fixing commit may contain the root-cause fix together with tests, wrappers, refactoring, or unrelated cleanup. The same fix may be backported to several release branches with different commit hashes, or applied as an equivalent fix that does not match the original patch text. Some CVEs require multiple fixes, receive partial fixes, or cross branch boundaries through cherry-picks and merges. Add-only fixes are also common: the patch adds a guard or validation check, but the vulnerable old statement is not deleted. Refactoring can move the same vulnerability condition across files, and a later release may contain similar code but a different guard or reachable sink. In these cases, the old code, the fixing commit, and the released-version set are connected by history, branch structure, and vulnerability conditions, not by a single textual range.

Existing work gives important evidence but does not fully close this gap. The affected-version benchmark by Chen et al. separates vulnerability-level exact match from version-level precision and recall, showing that partial success on individual tags does not imply recovery of the complete released-version set [1]. Affected-version methods such as V-SZZ, developer-log based analysis, and Vercation connect fixing commits, vulnerable code, and release versions more directly than advisory-only reasoning [2]–[4]. SZZ, LLM-assisted SZZ, agent-assisted SZZ, MAS-SZZ, and temporal-graph search improve how vulnerable statements and vulnerability-inducing commits are found [5]–[8]. Matching-based and recurring-vulnerability detectors provide scalable evidence about vulnerable code, patch signatures, and reused vulnerable components [9]–[12]. However, a bug- or vulnerability-inducing commit is a historical event, while an affected-version set is a release-tag-level vulnerability state. Code presence and patch resemblance are also evidence, not final affected-version decisions.

VulnVersion moves from single-BIC localization to vulnerability-state reconstruction for affected-version identification. Instead of forcing a CVE history into one inducing commit and one fixing commit, VulnVersion represents the history as role-labeled history events. These roles include introduction, prerequisite, refactoring, fix-series, boundary, unrelated, and uncertain events. The roles define how vulnerability state appears, persists, moves, and disappears over the Git DAG. The propagated state is then projected to official release tags to produce the affected-version set.

VulnVersion takes CVE metadata, fixing commits, and repository history as input. It first organizes attacker-relevant vulnerability conditions in an attacker-condition knowledge graph, including attacker-controlled input, trigger condition, exploit precondition, reachable sink, missing guard, unsafe state, and impact path. These conditions guide root-cause reasoning, pre-fix anchor selection, and history-event judgment. VulnVersion then reconstructs evidence-constrained history

events, judges boundary events, propagates vulnerability state over the Git DAG, and performs release tag projection. The method does not ask an LLM to issue independent verdicts for every release tag. Semantic judgment is scoped to repository evidence, candidate history events, and boundary decisions. Finally, vulnerability-pattern-driven self-evolution accumulates reviewed evidence, failure modes, anchor-selection outcomes, Judge decisions, and reusable priors across related CVEs. This feedback is evidence-gated: it is tied to provenance and verified evidence, not free-form model memory.

This paper makes the following contributions.

- **From BIC to vulnerability state for affected-version identification.** We formulate affected-version identification as vulnerability-state reconstruction rather than single-BIC localization. VulnVersion models a CVE history as role-labeled history events, including introduction, prerequisite, refactoring, fix-series, boundary, unrelated, and uncertain events. These event roles define how vulnerability state propagates over the Git DAG before being projected to release tags.
- **Attacker-condition knowledge graph.** We introduce an attacker-condition knowledge graph that structures attacker-controlled inputs, trigger conditions, exploit preconditions, reachable sinks, missing guards, unsafe states, and impact paths. These conditions guide root-cause reasoning, pre-fix anchor selection, and history-event judgment with source-level evidence.
- **Vulnerability-pattern-driven self-evolution.** We design an evidence-gated self-evolution mechanism that accumulates reviewed evidence, failure modes, anchor-selection outcomes, Judge decisions, and reusable priors across CVEs sharing repositories, CWEs, root-cause patterns, patch patterns, or attack behaviors. The accumulated patterns are reused as provenance-backed priors for later analyses.

We evaluate VulnVersion on the public affected-version benchmark used by prior work and report vulnerability-level Exact Accuracy, No-Miss Rate (NMR), and version-level micro precision, recall, and F1 [1]. [*Fill in Exact Accuracy, NMR, and version-level micro F1 after the full benchmark run.*] The evaluation also studies where failures arise across root-cause modeling, anchor selection, history reconstruction, boundary judgment, Git-DAG propagation, and release tag projection. We further analyze sensitivity to patch type, modification scope, and branch context, and evaluate the attacker-condition knowledge graph and vulnerability-pattern-driven self-evolution through ablations. [*Report the attacker-condition knowledge graph ablation after the full benchmark run.*] [*Report the vulnerability-pattern-driven self-evolution ablation after the full benchmark run.*] The rest of this paper is organized as follows. Section II defines the task and problem boundaries. Section IV presents the VulnVersion approach. Section VI describes the evaluation design. Section VII discusses limitations and threats to validity. Section VIII reviews related work.

II. Background and Problem Formulation

A. Affected-Version Identification

This paper studies the vulnerability-affected version identification task defined by the benchmark study of Chen et al. [1]. Given a reported vulnerability c, such as a CVE, a target project or repository P, a set of released versions or release tags $V = \{v_1, \ldots, v_n\}$, and patch evidence such as one or more fixing commits, the task is to identify the subset of releases in which the vulnerable logic remains present:

$$V_{aff}(c, P) = \{v \in V \mid vuln\_state(c, P, v) = \mathsf{true}\}.$$

Here $vuln\_state$ is a task-level predicate, not an assumed oracle. It names the conceptual release-level property that tag v still contains the vulnerability condition associated with c. Later sections operationalize this property through repository evidence and branch-specific history reasoning.

The output is therefore a set of released versions, not a vulnerability-inducing commit, not a vulnerable function, and not a textual advisory range. This distinction is central to the benchmark study by Chen et al.: the evaluation treats exact recovery of the full affected-version set as a vulnerability-level objective, while also measuring partial correctness over individual CVE-version pairs [1]. A method can be useful at one granularity and still fail at the other. For example, predicting many truly affected tags may yield a reasonable version-level recall, but a single omitted affected tag makes the CVE-level exact set wrong.

The benchmark setting is also repository-grounded. The released-version universe is determined by project release tags, and the ground-truth labels are attached to those released versions. In this paper, we use the same task framing and benchmark family as Chen et al.: the goal is to predict `affected_versions` for each CVE in the dataset, using repository and patch evidence rather than treating public advisory metadata as sufficient ground truth.

B. From Fixing Patches to Vulnerability State

A fixing patch is the most useful starting signal for this task, but it is not the answer. A commit that closes a CVE may contain the root-cause fix, supporting checks, error-handling changes, tests, documentation, refactoring, wrapper changes, formatting edits, or branch-specific backport adjustments. The patch also describes a transition from one repository state to another, whereas affected-version identification asks whether many released states still satisfy the vulnerable condition.

We use two conceptual predicates to separate these roles. A *root-cause predicate* describes the condition that makes the vulnerability possible in a repository state, such as a missing bounds check, an unchecked object lifetime, an attacker-controlled value reaching a sensitive operation, or an invalid state transition. A *fix predicate* describes the condition introduced or restored by the patch that prevents the vulnerability. Code presence alone may also be insufficient when the vulnerability depends on attacker-controlled inputs, trigger conditions, configuration, reachable sinks, or exploit preconditions. These attacker-relevant conditions should be preserved

as structured evidence rather than only as unstructured prompt text. These predicates are conceptual objects in this section; the concrete extraction and checking procedures are deferred to Section IV.

This predicate view prevents a common category error. The old side of a patch may provide a pre-fix anchor, but the anchor is only evidence about where to search. The new side of a patch may provide a fix predicate, but the absence of that new code in an old release does not automatically prove that the old release is vulnerable. A release is affected only when the root-cause predicate, under the relevant repository context, still holds in that release.

C. Attacker Conditions and Vulnerability Evidence Graphs

Affected-version identification cannot be reduced to code similarity, patch-line matching, or BIC localization. A vulnerable line matters only when the surrounding repository state still permits the vulnerable behavior. This state often depends on attacker-relevant conditions: attacker-controlled input, a trigger condition, an exploit precondition, a reachable sink, a missing guard, an unsafe state transition, or a security condition such as object lifetime, bounds, recursion, integer overflow, or a type invariant.

For this reason, a root-cause predicate is not just a code line. It should be tied to the vulnerable condition, the fix predicate or fix condition, source evidence, patch evidence, and the path or state constraint that makes the vulnerability relevant. The evidence may include a reachable sink, a missing guard, an unsafe state transition, a pre-fix anchor, or a history event that preserves or changes the condition. For affected-version identification, an attacker perspective does not mean proving a working exploit for every release. It means preserving the conditions under which the vulnerable behavior can be triggered, so that later history reconstruction does not confuse a syntactically similar line with a security-relevant state.

These relations are hard to preserve as flat prompt text. They are also easy to lose in a flat JSON record that lists evidence items without their roles. The evidence for one CVE can connect a CVE description, CWE category, fixing commit, patch hunk, pre-fix anchor, root-cause predicate, fix predicate, attacker condition, history event, and boundary event. A graph representation provides a structured substrate for these relations. In this limited sense, an attack-perspective knowledge graph is a way to organize attacker conditions and repository evidence, not a generic security ontology or an exploit proof. Its purpose is narrower: to keep evidence connected across the affected-version pipeline, from root-cause evidence to SZZ anchors, history events, boundary-event judgment, Git-DAG state propagation, and release tag projection.

D. Why BIC Localization Is Insufficient

SZZ-style methods locate bug-inducing commits (BICs) or vulnerability-inducing commits (VICs) by tracing from a fixing commit back through file history. This evidence is useful because the introduction of a vulnerable statement or state can bound where the vulnerability may have appeared. V-SZZ connects this idea to affected-version inference by mapping vulnerability-inducing commits to released version ranges [2]; later work improves commit localization through semantic analysis, LLM-assisted statement and candidate assessment, agentic repository search, and temporal knowledge-graph search [5], [6], [8], [13], [14].

However, a BIC or VIC is a history event, while an affected-version set is a projection over released repository states. Even a correct inducing commit does not by itself determine all affected releases. The method must still determine which branch-specific states reach official release tags, which branches received the fix, whether equivalent fixes exist without the same commit hash, and whether later code movement or partial reverts changed the vulnerability state.

The gap becomes larger in non-linear histories. A vulnerability may be introduced on one branch, fixed on another, backported to selected stable branches, or fixed through different commits in different release lines. Multi-fix CVEs and partial fixes further break the assumption that a single inducing commit and a single fixing commit define a closed linear interval. Therefore, BIC localization provides history evidence, but affected-version identification requires boundary-event judgment and branch-aware release projection.

E. Tracing-Based and Matching-Based Evidence

The benchmark study organizes existing affected-version tools into two evidence families [1]. Tracing-based methods start from fixing patches, select statements or hunks, trace them through history toward candidate inducing commits, and then infer affected versions from the resulting history range. Matching-based methods construct vulnerability signatures, patch signatures, or code representations and search historical versions for matching vulnerable logic.

Both families produce useful evidence, but neither evidence family is equivalent to complete affected-version reasoning. Tracing evidence can identify plausible historical events, yet it may be sensitive to statement selection, one-step blame assumptions, over-tracing, and weak cross-branch patch reuse. Matching evidence can detect code presence, yet it may be brittle under renaming, refactoring, minor syntactic edits, and semantically equivalent but syntactically different code.

The benchmark evaluation deliberately uses both vulnerability-level and version-level metrics because the two evidence families fail in different ways [1]. Tracing-based tools often preserve temporal context but can miss the true vulnerable condition when the selected patch lines are not root-cause lines. Matching-based tools can achieve high precision on exact or near-exact patterns but often lack semantic verification that a matched snippet still constitutes the vulnerability. For this reason, the present paper treats tracing and matching outputs as evidence sources rather than final affected-version decisions.

F. History and Release Structure Challenges

Affected-version identification is difficult because patch structure and release structure interact. Add-only patches are

hard for deletion-dependent tracing because the patch introduces guards or checks without deleting the vulnerable old statement. Deleted-line or modified-line patches are easier to anchor, but the deleted text may still be an implementation symptom rather than the root cause. Multi-function and multi-file patches add unrelated edits, helper changes, tests, or refactoring noise, making it harder to decide which code elements should drive the affected-version judgment.

Branch and release structure adds another layer. Many projects maintain multiple release lines, and a CVE fix may be applied to each line through cherry-picks, backports, merge commits, or semantically equivalent commits. A branch may receive only part of a fix, may revert a fix, or may already contain a different mitigation. Consequently, the same CVE can correspond to several boundary events: the first commit that introduces the vulnerable state, the fixing commit on one branch, equivalent fixes on other branches, and prerequisite commits that make the vulnerability reachable.

Code evolution also weakens naive signature reasoning. A vulnerable function can be renamed, split, copied, moved across files, wrapped by a new API, or refactored while preserving the same vulnerable behavior. Conversely, a syntactically similar function can be safe because a guard, configuration constraint, type invariant, or control-flow condition changed. The release universe itself must also be explicit. Affected-version predictions are evaluated over official released versions or release tags, not arbitrary Git refs or every historical tag.

G. Evidence-Gated Continual Adaptation

Affected-version identification has repeated structure across CVEs. Cases from the same repository may share release branches, backport style, subsystem layout, parser code, or dependency code. Cases with the same CWE may share a root-cause pattern, such as bounds checks, object lifetime, recursion limits, integer overflow, or type confusion. Fixing commits may also share patch patterns, such as add-only guards, changed validation logic, changed error paths, or branch-specific backports.

These repeated structures can help later CVEs, but only as evidence. They can inform pre-fix anchor selection, candidate history event ranking, boundary-event judgment, branch-specific fix reasoning, and failure diagnosis. For example, a verified observation that one repository backports fixes through equivalent stable-branch commits can help rank future candidate fixes in that repository. A verified failure mode for add-only guard patches can help avoid relying only on deleted lines. In both cases, the reusable information is the inspected evidence and its provenance, not a model's unsupported memory of a prior answer.

Direct model memory is risky for this task. It may hallucinate a prior pattern, overfit to development cases, preserve a wrong prior, leak benchmark labels, or hide the source of an error. Continual adaptation is useful only if it is evidence-gated. A previous decision should not become reusable knowledge merely because a model produced it. It should become reusable only when the decision is linked to inspected repository evidence, a stable patch pattern, or a verified failure mode.

Evidence-gated memory should therefore keep provenance, link observations to repository, CWE, and patch-pattern evidence, and allow an observation to be invalidated or downgraded when later evidence contradicts it. It should also separate reusable evidence from ground-truth affected-version labels. In this paper, continual adaptation is a design requirement and a future ablation target unless supported by explicit experiments. It should act as a prior for history reconstruction, not as the final affected-version decision.

H. Problem Statement

The problem studied in this paper is: given a CVE c, its fixing commit family F_c, a target repository P, and the official release-tag universe V, identify all and only release tags in V whose repository states satisfy the root-cause vulnerability predicate for c. Here, F_c denotes one or more fixing commits associated with the CVE, including branch-specific or equivalent fixes when such evidence is available. The expected output is a set of affected release tags, together with evidence and uncertainty records that explain how the prediction was derived.

This formulation deliberately excludes several adjacent tasks. It is not exploit generation and does not attempt to prove runtime exploitability for every released tag. It is not merely advisory normalization because advisory ranges can be incomplete, stale, or inconsistent with repository history. It is not only BIC or VIC localization because inducing commits are history events rather than release-level exposure labels. It is also not only vulnerable-code clone detection because code similarity is evidence, not a vulnerability-state verdict.

We describe the repository history as a Git directed acyclic graph (Git DAG). A *history event* is a commit-level change that may introduce, remove, preserve, or transform part of the vulnerability condition. A *boundary event* is a history event that changes the vulnerability state for a branch or release line. A *release tag projection* maps these branch-specific vulnerability states onto the official release-tag universe. These definitions provide the vocabulary needed by the later method section without committing this section to a particular algorithm.

I. Requirements for a Robust Solution

The preceding formulation implies that an effective solution must be root-cause-grounded. It must distinguish vulnerability-relevant evidence from incidental patch edits, and it must represent both the vulnerable condition and the fixing condition before projecting branch-specific states to release tags. Without this separation, a method can confuse a wrapper change, a test update, or a backport artifact with the actual security state.

The solution must also be evidence-constrained. Pre-fix anchors, history events, and boundary events should be accepted only with repository evidence that can be inspected

later. In particular, the method should separate the judgment that a candidate history event is relevant from the projection that converts branch-specific states into affected release tags. This separation is necessary because a candidate commit can be useful evidence without being the final affected-version boundary.

Finally, the solution must be branch-specific and auditable. It should reason over the Git DAG, release lines, backports, equivalent fixes, and official release tags rather than assuming a single linear version order. If LLMs or agents are used for semantic judgment, their role should be bounded and accountable: prompts, evidence inputs, uncertainty, failures, and decision artifacts must be recorded so that affected-version predictions can be inspected, debugged, and evaluated under the benchmark metrics. A robust solution should also support evidence-gated accumulation of repeated failures and verified history-event decisions as reusable repository, CWE, and patch-pattern evidence. Such memory must be auditable, not free-form model memory.

III. Motivating Example

IV. VulnVersionApproach

This section describes the current high-level design of VulnVersion. It is intentionally written as a structured placeholder: it captures the design choices that are already supported by the local SystemDesign documents, while leaving implementation details, optimization choices, and final algorithmic claims as [*NEEDS METHOD DETAIL*].

A. Overview

VulnVersiontakes as input a CVE record, the target repository, the fixing commit family, release tags, and optional CVE context such as NVD description or advisory text. It outputs a predicted set of affected release tags. The framework is organized into three stages: leftmargin=*

1) **Step1: fix-family and patch semantic filtering.** Parse fixing commits, extract changed files and hunks, classify commit-level and chunk-level roles, and produce patch semantics for later reasoning.
2) **Step2: root-cause vulnerability evidence extraction.** Convert patch semantics and CVE context into root-cause-level evidence, including vulnerable sequences, fix guards, scope constraints, and evidence risks.
3) **Step3: boundary-event reconstruction and release projection.** Build release-line structure, judge boundary events, propagate vulnerability state through the Git DAG, and project affected states to release tags.

The corpus synthesis suggests a design rule that should be preserved as the method matures: deterministic evidence preparation should reduce the search space before expensive semantic judgment. FIRE, MOVERY, ReDeBug, V1SCAN, VULTURE, and VUDDY all use staged filtering or indexing to make vulnerability search scalable [9]–[12], [15], [16]; AgentSZZ and related agent papers show that scoped tool interfaces and context compression are important for keeping LLM-based repository investigation tractable [6], [14]. VulnVersionadopts this as a paper-level boundary: semantic judgment should be scoped to evidence-backed boundary events, while Git-DAG state propagation and release-tag projection remain deterministic.

B. Step1: Fix-Family Semantic Filtering

The first stage exists because a CVE may map to multiple fixing commits and because each fixing commit may contain irrelevant hunks. The local design document records that the baseline dataset contains 1,128 CVEs and 1,542 patches, with 68 multi-commit CVEs after local flattening. It also records 4,783 patch chunks across the local dataset processing artifacts. These figures should be rechecked before camera-ready use [*NEEDS EVIDENCE*].

Step1 is designed to output structured artifacts such as `fix_family_semantics.json`, `commit_semantics.jsonl`, `chunk_semantics.jsonl`, and `patch_semantics.json`. Its role is not to decide affected versions. Instead, it provides clean, typed, and source-referenced patch evidence to Step2. In particular, Step1 should distinguish primary fixes from supporting fixes, contextual changes, tests, documentation, wrapper commits, and refactoring noise.

C. Step2: Root-Cause-Level Evidence

Step2 extracts a root-cause-level vulnerability existence theorem, abbreviated as VET in the design documents. The intended VET representation separates five concepts:

$$\Theta = \langle S, V, F, G, C \rangle$$

where S is the vulnerability scope, V is vulnerable evidence, F is fix evidence, G is guard or applicability constraints, and C is the certificate policy. This representation is still a design placeholder [*NEEDS METHOD DETAIL*], but it encodes an important boundary: touched files and changed lines are not automatically root-cause evidence.

Step2 should output artifacts such as `root_cause_vet.json`, `vet_evidence_graph.json`, `vet_admission_input.json`, and a compatibility `rci.json`. These artifacts provide the evidence packet used by Step3 for prioritizing boundary-event reconstruction.

This design is motivated by repeated warnings across the reference corpus: patch signatures, deleted lines, and cloned functions are useful evidence, but none of them alone proves that the vulnerability condition holds in a released version [2], [4], [10], [11], [15]. The VET artifact is therefore meant to make the required vulnerable condition explicit before boundary events are judged.

D. Step3: Boundary Reconstruction and Release Projection

Step3 is responsible for affected-version identification. The current main path is: fixing commits, candidate history events, boundary-event judgment, Git-DAG state propagation, and

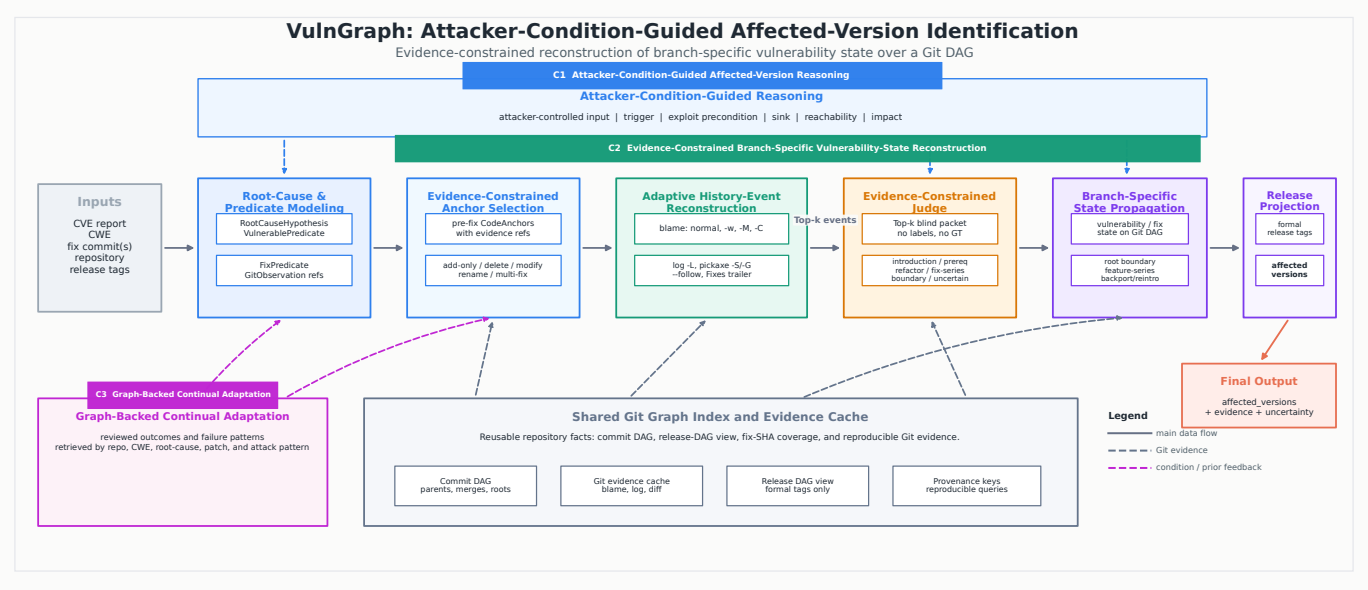

Fig. 1. Overview of VulnGraph.

release-tag projection. The semantic-judgment boundary is explicit: judgment is applied to candidate boundary events, while the release-line graph and release-tag projection are handled by deterministic repository analysis.

This separation is central to VulnVersion. Deterministic Git-DAG state propagation keeps the method auditable and cost-aware; semantic judgment is reserved for cases where boundary evidence must be interpreted. The final Step3 design still requires experimental validation before it can be presented as a complete algorithm [*NEEDS EXPERIMENT*].

E. Expected Agent Interface

The planned semantic-judgment interface is deliberately narrow. For each candidate boundary event, the model receives scoped evidence derived from Step1 and Step2, repository snippets around the event, and a required output schema containing a selected, rejected, or uncertain judgment plus evidence references. Inspired by agentic SZZ evaluations [6], [14], VulnVersionshould log token usage, tool calls, latency, and failure modes for every judgment. The exact prompt, tool API, and judgment schema remain [*NEEDS METHOD DETAIL*].

F. Memory and Self-Evolution Hooks

The broader SystemDesign materials describe typed memory, verifier-gated skill evolution, and artifact promotion as future capabilities. In this draft, these are not claimed as completed experimental components. They are positioned as optional design hooks: failed boundary judgments may become failure memories, repeated repair procedures may become procedural memories, and stable verified patterns may eventually be compiled into deterministic artifacts. Any final claim about these mechanisms requires implementation and ablation evidence [*NEEDS EXPERIMENT*].

V. Implementation

VI. Evaluation Design

This section specifies the planned evaluation. It does not report final VulnVersionresults. All missing results are explicitly marked.

A. Dataset

The main dataset will be the benchmark used by "Vulnerability-Affected Versions Identification: How Far Are We?" [1]. The local reference analysis states that the benchmark contains 1,128 C/C++ vulnerabilities from nine open-source projects and 132 vulnerability types. Exact project-level table values and patch-type counts must be verified against repaired tables before final use [*NEEDS TABLE REPAIR*]. The dataset is selected because it is directly aligned with our target problem: given CVEs and patch commits, identify affected release versions.

B. Baselines

We will compare against the baselines organized in the benchmark study: leftmargin=*

- **Tracing-based:** VCCFinder [17], V-SZZ [2], Lifetime [18], SEM-SZZ [19], TC-SZZ [13], and LLM4SZZ [5].
- **Matching-based:** ReDeBug [15], VUDDY [11], MOVERY [10], V1SCAN [16], FIRE [9], and VULTURE [12].

If official runnable artifacts are unavailable or inconsistent, we will report whether baseline values are taken from the paper table or reproduced locally. No reproduced baseline claim will be made until scripts, data, and environment are verified [*NEEDS ARTIFACT*].

C. Metrics

We follow the metric structure extracted from the baseline and related papers: leftmargin=*

- **CVE-level Accuracy:** the fraction of CVEs for which the predicted affected-version set exactly equals the ground truth set.
- **CVE-level No-Miss Ratio:** the fraction of CVEs for which all ground-truth affected versions are included in the prediction.
- **Version-level Precision/Recall/F1:** flattened version-level metrics over all CVE-version pairs.
- **Resource metrics:** average time, input tokens, output tokens, total tokens, cost, agent calls, candidate boundary events, and execution failure rate for VulnVersion[*NEEDS EXPERIMENT*].
- **Manual-workload and lower-bound metrics:** if complete FN validation is infeasible for some cases, we will report conservative lower-bound recall and manual inspection counts, following the evaluation style of developer-log based affected-version work [3].

D. Research Questions

RQ1: Overall effectiveness. How accurately does VulnVersionidentify affected versions compared with tracing-based and matching-based baselines? We will report CVE-level Accuracy, CVE-level NMR, and version-level precision/recall/F1. Result table: [*NEEDS EXPERIMENT*].

RQ2: Robustness across patch and repository conditions. How does performance vary across add-only, delete-only, mixed patches, single-file and multi-file patches, and single-branch versus multi-branch settings? Subset definitions will follow the benchmark where available. Result table: [*NEEDS EXPERIMENT*].

RQ3: Contribution of each VulnVersionstage. What is the effect of patch-family filtering, root-cause evidence extraction, boundary-event judgment, and Git-DAG state propagation? Planned ablations include disabling Step1 filtering, replacing Step2 VET with raw touched-code evidence, replacing repository-aware release projection with simpler tag ordering, and replacing boundary-event judgment with deterministic matching. Result table: [*NEEDS EXPERIMENT*].

RQ4: Cost and failure analysis. What is the runtime, token cost, and failure rate of VulnVersion? Which cases fail because of patch noise, root-cause extraction error, boundary-event judgment error, release projection error, timeout, or missing artifacts? Result table: [*NEEDS EXPERIMENT*].

RQ5: Agent behavior and evidence use. When VulnVersioninvokes an agent, which evidence sources and tool calls drive semantic judgments, and how do candidate-set size, token usage, and cost correlate with success? This RQ is inspired by recent agentic SZZ analyses that report tool calls, tokens, cost, and behavior distributions [6], [14]. Result table: [*NEEDS EXPERIMENT*].

E. Pilot Status

The local SystemDesign directory contains current pilot artifacts over a 32-case subset. These artifacts are useful for debugging the paper pipeline and metric implementation, but they are not the final ICSE results and should not be compared directly against full 1,128-CVE paper baselines. Any pilot table copied into the paper must be labeled as a pilot and kept separate from the main evaluation [*NEEDS EXPERIMENT*].

VII. Threats to Validity

Construct validity. The affected-version task has both exact-set and partial-version aspects. We therefore use CVE-level Accuracy, CVE-level No-Miss Ratio, and version-level precision/recall/F1 rather than relying on a single metric. Resource metrics such as tokens and cost must be reported separately because an agent-based method can trade accuracy for runtime or API cost.

Internal validity. VulnVersiondepends on correct patch-family filtering, root-cause evidence extraction, boundary-event judgment, and Git-DAG state propagation. Errors in any stage can propagate to the final affected-version set. The planned ablations in RQ3 are necessary before attributing improvements to a specific component. Current method details and results remain [*NEEDS EXPERIMENT*].

External validity. The primary dataset is C/C++ focused and covers nine open-source projects. Results may not transfer directly to ecosystems with different release practices, package managers, or vulnerability disclosure conventions. We will avoid claims about other languages until additional datasets are evaluated.

Baseline validity. The baseline study provides the most relevant comparison point, but local reproduction depends on artifact availability, environment compatibility, and exact data splits. If we use paper-reported baseline values, we must clearly label them as paper-reported. If we reproduce them, we must release scripts and environment details [*NEEDS ARTIFACT*].

Citation and table validity. Several local reference analyses mark table values, figure crops, and citation metadata as requiring repair or verification. Therefore, all exact numeric claims taken from extracted tables must be checked against the original PDFs before submission [*NEEDS TABLE REPAIR*] [*NEEDS CITATION VERIFICATION*].

Agent validity. If VulnVersionuses an LLM or agent backend, semantic judgments may depend on prompt design, context compression, model version, randomness, and tool-output formatting. We will log model/backend information, tokens, cost, latency, failure rate, and replay artifacts where possible. Any claim about agent memory or self-evolution remains out of scope until supported by ablations [*NEEDS EXPERIMENT*].

Ethics and misuse. Affected-version identification can help defenders prioritize patching, but it can also reveal which deployed versions are likely vulnerable. Prior patch-search work explicitly surfaces this dual-use risk [15]. The paper should emphasize defensive use, benchmark-level evaluation,

and responsible disclosure if new vulnerable versions are discovered [*NEEDS EVIDENCE*].

VIII. Related Work

A. Affected-Version Identification

Affected-version identification takes a disclosed vulnerability and predicts the set of released versions in which the vulnerable logic remains present. Chen et al. formalize this task over a release universe and evaluate it at two granularities [1]. Vulnerability-level evaluation requires an exact match of the complete affected-version set. Version-level precision, recall, and F1 instead measure individual CVE-version decisions and therefore capture partial correctness. This distinction separates affected-version identification from vulnerability-inducing commit (VIC) localization and from advisory range normalization.

Direct methods use different forms of repository evidence. V-SZZ traces patch-related lines to candidate VICs, detects duplicated changes across branches, and maps inducing and fixing commits to version tags [2]. The patch-and-developer-log approach of He et al. organizes releases as a version tree and reasons about first and last affected versions, branch boundaries, partial matches, and repatching [3]. Vercation combines program slicing, LLM-based vulnerable-statement selection, and structural clone comparison before deriving affected versions between a VIC and a fixing commit [4]. A recent anonymous manuscript, *CaVulner*, also uses CVE/CWE context, LLM-assisted root-cause localization, blame, and duplicate-change analysis.[1]

These methods establish that patch semantics, commit history, duplicated fixes, and release structure all matter. However, a VIC, a fixing commit, or a version-range endpoint does not by itself represent partial fixes, equivalent fixes, reversions, and multiple branch-specific state changes. VulnVersion therefore targets complete released-version set prediction through history event reconstruction, boundary judgment, Git-DAG vulnerability-state propagation, and release tag projection.

B. SZZ and Agent-Assisted History Reconstruction

SZZ research improves the evidence used to locate bug-inducing commits (BICs) or VICs. Early approaches derive candidates from lines changed by a fixing commit. VCCFinder uses blame-derived labels together with code and repository features to prioritize suspicious commits, while Lifetime uses weighted blame evidence to estimate vulnerability lifetimes [17], [18]. Semantic variants refine which statements should be traced. SEM-SZZ compares local program states and data-flow evidence; TC-SZZ performs iterative history tracing and studies cases that direct blame cannot recover; and differential patch-pattern analysis extracts vulnerability-critical statement sequences before matching earlier commits [13], [19], [20]. These methods improve candidate evidence, but their main output remains a commit or a lifetime boundary.

[1]Citation gap: the available CaVulner manuscript is anonymous and has no stable publication metadata.

LLM-assisted methods replace fixed heuristics with semantic assessment. LLM4SZZ ranks buggy statements and evaluates candidate commits with expanded context [5]. AgentSZZ adds task-specific repository tools, domain knowledge, and iterative exploration for ghost and cross-file cases [6]. Other agentic workflows search file histories through binary or direct candidate selection and derive short patterns from the fixing commit for searching candidate changes [14]. These approaches broaden repository exploration, but they are evaluated on BIC identification rather than complete affected-version set prediction.

MAS-SZZ is especially close to the evidence layer of VulnVersion. It summarizes a root cause with traceable evidence, infers patch-hunk intent, selects vulnerability-related anchor statements, and uses an agent to backtrack until it finds a VIC [7]. VulnVersion does not claim that evidence-grounded root-cause analysis or semantic anchor selection is unique. Its distinction begins after anchor selection: selected statements act as evidence-constrained pre-fix anchors for reconstructing history event chains and judging branch-specific introduction boundaries, rather than serving only as paths to a final VIC.

Beyond Blame is also closely related because it reframes BIC identification as agent-guided search over a temporal knowledge graph. Its graph expands candidates from blame commits and the fixing commit through temporal and structural commit relations [8]. VulnVersion shares the view that a single blame result is insufficient. The task and graph roles differ, however. Beyond Blame searches for BICs; VulnVersion reconstructs introduction, transformation, fix, and reversion events, then propagates vulnerability state through the Git DAG and projects that state to official release tags.

C. Matching-Based and Recurring Vulnerability Detection

Matching-based systems provide direct evidence that vulnerable or patched code occurs in a target codebase. ReDeBug uses normalized token windows and Bloom filters for scalable unpatched-clone search, while VUDDY indexes normalized function fingerprints [11], [15]. These designs provide efficient and precise evidence for exact or abstracted clones. Their function or token granularity, however, does not encode the full repository context in which a matched fragment is security-relevant.

Modification-aware systems recover more varied forms of reuse. MOVERY distinguishes vulnerability signatures from patch signatures, V1SCAN combines version-based filtering with code classification, and VULTURE combines version evidence with patch- and chunk-level analysis [10], [12], [16]. FIRE further uses staged clone filters followed by taint-based verification [9]. These methods should not be reduced to simple textual matching: they contribute normalized signatures, structural evidence, patch-state evidence, code classification, and taint paths.

Their remaining limitation is a task boundary. A detector centered on deleted pre-fix text has little direct evidence for an add-only guard. Exact and near-exact signatures can also lose matches after semantic transformation, code movement,

or branch divergence, although abstraction and modification-aware signatures mitigate some of these cases. Conversely, a code match does not establish that trigger conditions, guards, and reachable sinks are unchanged. Matching evidence is therefore valuable for candidate generation and boundary judgment, but it cannot alone reconstruct branch-specific affected versions across a Git DAG.

D. Security Knowledge Graphs and Attacker-Condition Reasoning

Security knowledge graphs commonly organize threat intelligence rather than repository evolution. ATT&CK-to-CVE graphs integrate CVE, CWE, CAPEC, and ATT&CK entities to support cross-source and multi-hop analysis [21]. NEXUS maps CVE descriptions to ATT&CK techniques and supports analyst feedback for later mapping decisions [22]. D3FEND models defensive techniques, artifacts, and their relations to offensive techniques, while KRYSTAL links audit provenance with background knowledge for attack detection and scenario reconstruction [23], [24]. These graphs answer questions about attack techniques, defenses, threat intelligence, or observed system events. They do not reconstruct affected release states from a fixing commit.

Beyond Blame uses a different kind of graph: a temporal graph of commits for BIC search [8]. This graph preserves temporal and structural repository relations, but it is not an attacker-condition graph. The distinction matters because an affected-version decision may depend on attacker-controlled input, an exploit precondition, a reachable sink, a missing guard, or an unsafe state even when similar code is present.

For VulnVersion, the attacker perspective defines which security conditions should be preserved as structured evidence during history reconstruction. It does not mean generating or executing an exploit for every release tag. The intended graph role is to connect attacker conditions and root-cause evidence to pre-fix anchors, history events, and boundary events. We do not claim in this paper that an attacker-condition knowledge graph has already improved affected-version results; that claim requires a dedicated evaluation.

E. Evidence-Gated Continual Adaptation

Recent agent research studies long-term memory, procedural memory, and reusable skills. Surveys organize the roles and risks of agent memory, while procedural-memory systems extract reusable action knowledge from prior trajectories [25], [26]. Self-evolving agent work further learns, verifies, or distills reusable skills from experience [27]–[30]. These directions motivate knowledge reuse across repeated software-analysis tasks.

Affected-version identification imposes a stricter evidence contract on such reuse. A prior decision should be reusable only when it is linked to verified repository evidence and records its provenance. Useful memory includes inspected failure patterns and repository-, CWE-, or patch-pattern priors. Ground-truth affected-version labels must remain separate from this memory, and contradictory evidence must be able to invalidate or downgrade an earlier observation. This evidence-gated continual adaptation differs from accepting free-form model memory as fact.

VulnVersion treats continual adaptation as a design boundary, not as an empirically validated contribution. Reused evidence may guide pre-fix anchor selection, history event ranking, and failure diagnosis, but it cannot replace boundary-event judgment or release tag projection. A dedicated ablation is required before any effectiveness claim is made.

IX. Conclusion

This draft frames VulnVersionas a repository-grounded approach to identifying CVE-affected released versions from patch evidence. The central position is that affected-version identification should be treated as evidence-constrained history reconstruction and release-tag projection, not only as vulnerability-inducing commit localization or patch-signature matching. The current paper skeleton establishes the task definition, motivation, related-work structure, method placeholder, and evaluation plan around the "How Far Are We?" benchmark. Final method details, full experimental results, ablations, resource measurements, and verified citation metadata remain required before this can become an ICSE submission [*NEEDS EXPERIMENT*].

References

[1] X. Chen, C. Liu, J. Cao, Y. Xiao, X. Cai, Y. Li, J. Shi, T. Sun, H. Chen, and W. Huo, "Vulnerability-Affected Versions Identification: How Far Are We?" 2025. [Online]. Available: https://arxiv.org/abs/2509.03876

[2] L. Bao, X. Xia, A. E. Hassan, and X. Yang, "V-SZZ: Automatic identification of version ranges affected by CVE vulnerabilities," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: ACM, 2022, pp. 2352–2364. [Online]. Available: https://doi.org/10.1145/3510003.3510113

[3] Y. He, Y. Wang, S. Zhu, W. Wang, Y. Zhang, Q. Li, and A. Yu, "Automatically identifying CVE affected versions with patches and developer logs," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 905–919, 2024. [Online]. Available: https://doi.org/10.1109/TDSC.2023.3264567

[4] Y. Cheng, T. Zhang, L. K. Shar, S. Yang, C. Dong, D. Lo, S. Lv, Z. Shi, and L. Sun, "VERCATION: Precise vulnerable open-source software version identification based on static analysis and LLM," *IEEE Transactions on Software Engineering*, vol. 52, no. 2, pp. 376–394, 2026. [Online]. Available: https://doi.org/10.1109/TSE.2025.3599581

[5] L. Tang, J. Liu, Z. Liu, X. Yang, and L. Bao, "LLM4SZZ: Enhancing SZZ algorithm with context-enhanced assessment on large language models," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 343–365, 2025. [Online]. Available: https://doi.org/10.1145/3728885

[6] Y. Lyu, J. Shi, H. J. Kang, R. Widyasari, J. He, Y. Niu, C. Yang, J. Chen, Z. Yang, J. Lawall, and D. Lo, "AgentSZZ: Teaching the LLM agent to play detective with bug-inducing commits," 2026. [Online]. Available: https://arxiv.org/abs/2604.02665

[7] S. Cao, J. Xu, L. Yu, J. Yang, X. Lin, L. Zhu, and F. Xiao, "MAS-SZZ: Multi-agentic SZZ algorithm for vulnerability-inducing commit identification," 2026. [Online]. Available: https://arxiv.org/abs/2604.24398

[8] Y. Shi, H. Li, B. Adams, and A. E. Hassan, "Beyond Blame: Rethinking SZZ with knowledge graph search," 2026. [Online]. Available: https://arxiv.org/abs/2602.02934

[9] S. Feng, Y. Wu, W. Xue, S. Pan, D. Zou, Y. Liu, and H. Jin, "FIRE: Combining Multi-Stage filtering with taint analysis for scalable recurring vulnerability detection," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1867–1884. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/feng-siyue

[10] S. Woo, H. Hong, E. Choi, and H. Lee, "MOVERY: A precise approach for modified vulnerable code clone discovery from modified open-source software components," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3037–3053. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/woo

[11] S. Kim, S. Woo, H. Lee, and H. Oh, "VUDDY: A scalable approach for vulnerable code clone discovery," in *2017 IEEE Symposium on Security and Privacy*. IEEE, 2017, pp. 595–614. [Online]. Available: https://doi.org/10.1109/SP.2017.62

[12] S. Xu, J. Dong, W. Cai, J. Li, A. Shaghaghi, N. Sun, and S. Ma, "Enhancing security in third-party library reuse: Comprehensive detection of 1-day vulnerability through code patch analysis," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '25. Internet Society, 2025. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/enhancing-security-in-third-party-library-reuse-comprehensive-detection-of-1-day-vulnerability-through-code-patch-analysis/

[13] Y. Lyu, H. J. Kang, R. Widyasari, J. Lawall, and D. Lo, "Evaluating SZZ implementations: An empirical study on the Linux kernel," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2219–2239, 2024. [Online]. Available: https://doi.org/10.1109/TSE.2024.3406718

[14] N. Risse and M. Böhme, "How and why agents can identify bug-introducing commits," 2026. [Online]. Available: https://arxiv.org/abs/2603.29378

[15] J. Jang, A. Agrawal, and D. Brumley, "ReDeBug: Finding unpatched code clones in entire OS distributions," in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 48–62. [Online]. Available: https://doi.org/10.1109/SP.2012.13

[16] S. Woo, E. Choi, H. Lee, and H. Oh, "V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 6541–6556. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/woo

[17] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, S. Fahl, and Y. Acar, "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '15. New York, NY, USA: ACM, 2015, pp. 426–437. [Online]. Available: https://doi.org/10.1145/2810103.2813604

[18] N. Alexopoulos, M. Brack, J. P. Wagner, T. Grube, and M. Mühlhäuser, "How long do vulnerabilities live in the code? A large-scale empirical measurement study on FOSS vulnerability lifetimes," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 359–376. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alexopoulos

[19] L. Tang, C. Ni, Q. Huang, and L. Bao, "Enhancing bug-inducing commit identification: A fine-grained semantic analysis approach," *IEEE Transactions on Software Engineering*, vol. 50, no. 11, pp. 3037–3052, 2024. [Online]. Available: https://doi.org/10.1109/TSE.2024.3468296

[20] Q. Guo and Y. He, "Accurate identification of the vulnerability-introducing commit based on differential analysis of patching patterns," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '26. Internet Society, 2026. [Online]. Available: https://doi.org/10.14722/ndss.2026.230140

[21] M. Rahman, M. Akbar, J. T. Aguayo, M. S. Hossain, T. M. Ellis, S. S. Das, M. F. Babar, M. Hasan, A. Sanchez, L. F. de la Torre, and M. Halappanavar, "ATT&CK to CVE: A large-scale automated knowledge graph for threat intelligence," in *2025 IEEE International Conference on Data Mining Workshops*, ser. ICDMW '25. IEEE, 2025, pp. 1044–1054. [Online]. Available: https://doi.org/10.1109/ICDMW69685.2025.00140

[22] E. Khodayarseresht, S. Majumdar, S. Mokhov, and M. Debbabi, "NEXUS: Towards accurate and scalable mapping between vulnerabilities and attack techniques," in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS '26. Internet Society, 2026. [Online]. Available: https://doi.org/10.14722/ndss.2026.242926

[23] P. E. Kaloroumakis and M. J. Smith, "Toward a knowledge graph of cybersecurity countermeasures," The MITRE Corporation, Tech. Rep., 2021. [Online]. Available: https://d3fend.mitre.org/resources/D3FEND.pdf

[24] K. Kurniawan, A. Ekelhart, E. Kiesling, G. Quirchmayr, and A. M. Tjoa, "KRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data," *Computers & Security*, vol. 121, p. 102828, 2022. [Online]. Available: https://doi.org/10.1016/j.cose.2022.102828

[25] Y. Hu, S. Liu, Y. Yue, G. Zhang, B. Liu, F. Zhu, J. Lin, H. Guo, S. Dou, Z. Xi, S. Jin, J. Tan, Y. Yin, J. Liu, Z. Zhang, Z. Sun, Y. Zhu, H. Sun, B. Peng, Z. Cheng, X. Fan, J. Guo, X. Yu, Z. Zhou, Z. Hu, J. Huo, J. Wang, Y. Niu, Y. Wang, Z. Yin, X. Hu, Y. Liao, Q. Li, K. Wang, W. Zhou, Y. Liu, D. Cheng, Q. Zhang, T. Gui, S. Pan, Y. Zhang, P. Torr, Z. Dou, J.-R. Wen, X. Huang, Y.-G. Jiang, and S. Yan, "Memory in the age of AI agents," 2025. [Online]. Available: https://arxiv.org/abs/2512.13564

[26] R. Fang, Y. Liang, X. Wang, J. Wu, S. Qiao, P. Xie, F. Huang, H. Chen, and N. Zhang, "Memp: Exploring agent procedural memory," 2025. [Online]. Available: https://arxiv.org/abs/2508.06433

[27] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, "MemSkill: Learning and evolving memory skills for self-evolving agents," 2026. [Online]. Available: https://arxiv.org/abs/2602.02474

[28] Y. Yang, B. Li, Q. Pan, B. Zhang, X. Cai, L. Du, J. Zhou, K. Chen, Q. Chen, X. Li, B. Zhang, and L. He, "AutoSkill: Experience-driven lifelong learning via skill self-evolution," 2026. [Online]. Available: https://arxiv.org/abs/2603.01145

[29] H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, X. Liu, X. Li, and P. S. Yu, "CoEvoSkills: Self-evolving agent skills via co-evolutionary verification," 2026. [Online]. Available: https://arxiv.org/abs/2604.01687

[30] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang, "Trace2Skill: Distill trajectory-local lessons into transferable agent skills," 2026. [Online]. Available: https://arxiv.org/abs/2603.25158